Report

- Tanishq Garg (1535295)
- Suhas Agrawal (1531129)

1. Approach

Our agent plays Cascade using Minimax search with Alpha-Beta pruning. The agent maintains an internal copy of the board state, updated after every turn, and uses this to search ahead and pick the best action available.

The game is divided into two phases, placement and play, each handled differently. During the placement phase, the branching factor is very high (up to 64 possible placements), so we search to a shallower depth of 2. During the play phase, where actions are more constrained, we search to depth 3. This distinction allows the agent to remain within the 180-second CPU time limit while still looking ahead usefully.

At each node in the search tree, the active player's legal actions are generated via get_legal_actions(), which enumerates all valid PLACE, MOVE, EAT, and CASCADE actions for the current board. The agent assumes the opponent plays optimally, alternating between maximising (our agent) and minimising (opponent) nodes in the standard Minimax fashion.

2. Alpha-Beta Pruning and Move Ordering

Our Minimax Implementation uses Alpha-Beta pruning to reduce the number of nodes evaluated without affecting the correctness of the result. To maximise the effectiveness of pruning, we sort actions before expanding them at each node using a deliberate ordering: EAT actions are evaluated first, CASCADE second, and MOVE last.

This ordering is based on strategic reasoning. EAT actions directly capture enemy tokens and almost always produce the highest heuristic scores, so evaluating them first sets a tight alpha bound early and causes weaker branches to be pruned immediately. CASCADE actions can eliminate multiple tokens and disrupt enemy positioning but with less certainty than a direct capture, so they follow. MOVE actions reposition or merge friendly stacks and rarely produce immediate score improvements, meaning they are evaluated last where they are most likely to be cutoff entirely.

The practical effect is that the agent explores the most promising lines of play first, this leads to much better pruning compared to processing actions in random order and allowing us to search effectively within the fixed depth limits.

We also avoid creating deep copies of the board at every node, which would be expensive. Instead, the board is mutated in place using apply_action() and undo_action(), avoiding the cost of copying the board repeatedly and keeping the search fast across all depths.

3. Evaluation Function

When the search reaches a leaf node, either at the depth limit or when the game is over, the board is scored using a static evaluation function from the perspective of our agent. The function combines three components, each with its own strategic purpose.

The first and most heavily weighted component is token advantage, calculated as the difference between our total token count and the opponent's. This captures the core objective of the game: having more tokens on the board than your opponent. A win state (opponent has zero tokens) returns positive infinity, and a loss state returns negative infinity, ensuring the agent always aims for a win rather than a small material advantage.

The second component rewards eat threats. For every enemy stack that our agent can immediately capture on the next turn, a bonus proportional

to the enemy stack's height is added to the score. This encourages the agent to seek out attacking positions rather than just preserving its own material, since being adjacent to a capturable enemy is useful even before the capture happens.

The third component is an edge penalty. Our own stacks sitting on the board perimeter are penalised proportional to their height. Perimeter stacks are vulnerable to being cascaded off the board entirely, so the agent is discouraged from placing or moving pieces to the edges unless there is a strong reason to do so.

Together these three components give the agent a reasonable sense of who is winning at any given board state, balancing material count, attacking potential, and positional safety.

4. Placement Strategy

During the placement phase, the branching factor is at its highest since any empty cell on the board is a legal placement. To address this, we designed a function called _placement_candidates() that restricts the set of considered placements to a small, focused subset based on strategy rather than all empty cells.

The function considers two sets of cells. The first is the central 4x4 region of the board. Central positions are generally stronger in Cascade because they allow stacks to attack, move and cascade in more directions as compared to the ones on or near the edges. The second set is all empty cells within two steps of any existing friendly stack. Placing near your own pieces enables future merges and coordinated attacks, whereas isolated placements scattered across the board are difficult to coordinate effectively.

A fallback is also included so that if neither set yields any candidates, all empty cells are considered. This prevents the agent from getting stuck in edge cases early in the game.

This function is active in the final implementation and directly reduces the branching factor during placement turns. Rather than spending search budget on cells that are unlikely to lead anywhere useful, the agent focuses on positions that actually matter strategically, which makes the search much more focused.

5. Performance Evaluation

We evaluated our agent in two ways: through self-play using the provided referee program, and through submission to the Gradescope autograder against the provided test opponent.

In self-play we ran three games of the agent against itself, out of which the agent won twice. The first game ended in a win for Blue after 72 turns. The second win came in 51 turns for Red. The last game ended in a draw after 135 turns. In this game both agents reached an endgame state with only one or two tokens each on opposite sides of the board, and with no captures available, both sides fell into a repetition loop that the game eventually resolved as a draw.

On Gradescope, our agent was tested against the provided test opponent in both colour configurations. It won convincingly as Red in 37 turns and as Blue in just 22 turns, with the final board states showing the agent completely dominating the board. These results suggest the agent performs reliably against weaker opponents and that the evaluation function and move ordering are working as intended.

We did not implement a separate weaker baseline agent to compare against, so our performance evaluation is limited to these tests. A useful next step would be to pit different versions of the agent against each other, for example with and without move ordering, to better understand what each part actually contributes.

6. Limitations and Future Work

While the agent performs well against weaker opponents, there are several known limitations that affect its play in more complex situations.

The most visible issue is the move repetition in the endgame. When a few tokens remain on the board and no captures are immediately available, the agent can fall into a cycle of moving the same piece back and forth indefinitely. This happens because the fixed depth search has no memory of previously visited states and cannot detect that it is repeating the same position. Adding a transposition table, which stores previously evaluated board states and their scores, would address this directly and also speed up the search by avoiding repeated work.

A second limitation is the use of fixed search depths. The agent searches to depth 2 during placement and depth 3 during play regardless of how much time remains. This means the agent often finishes its turn well within the time limit without using the remaining time usefully. Iterative deepening would allow the agent to search progressively deeper and return the best result found when time runs out, making full use of the 180 second limit.

Finally, the evaluation function weights were set manually based on intuition rather than through any proper testing process. Testing different weight combinations through repeated self-play would likely improve the agent's strategic judgement.

References

Russell, S. and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. Chapter 5 (Adversarial Search and Games) provided the theoretical basis for the Minimax algorithm and Alpha-Beta pruning used in this project.